DNN Obfuscation With Contrastive Learning for Efficient Defense Against Side-Channel Attacks

Yidan Sun, Guiyuan Jiang, *Member, IEEE*, Jiayang Liu, Qian Sun, Peilan He, and Siew-Kei Lam, *Senior Member, IEEE*

***Abstract*—Deep Neural Networks (DNNs) have achieved notable advancements across various domains. Yet, they remain vulnerable to Side-Channel Attacks (SCAs) which expose them to model theft, posing substantial security threats. Existing countermeasures for mitigating SCAs using DNN obfuscation often involve impractical hardware modifications or suffer from poor scalability due to the need for time-consuming deployment on target platforms. We introduce CLOb-DNN (Contrastive Learning Obfuscation for DNNs), a novel contrastive learning-based framework that improves both the efficiency and robustness of DNN obfuscation against SCAs. By leveraging triplet networks, we effectively reduce the need for repeated physical deployments, substantially improving obfuscation efficiency while ensuring that the obfuscated DNNs meet latency constraints. Our framework integrates architecture-level and scheduling-level obfuscation to enhance stability and avoid costly modifications to backend source codes. We perform extensive evaluations using multiple state-of-the-art attack models to demonstrate the robustness of our approach against diverse threats. Experiments show that our method reduces obfuscation time by approximately 90%, achieves superior defense capability, and successfully meets various latency requirements, thereby offering a practical and effective solution for protecting DNN IPs against SCAs.**

***Index Terms*—DNN obfuscation, side-channel attacks, model IP protection, architecture and scheduling obfuscation.**

I. Introduction

RECENT years have seen rapid advances in Deep Neural Networks (DNNs), leading to major breakthroughs in domains such as computer vision [1], [2], natural language processing [3], [4], and time series forecasting [5], [6]. These valuable DNN intellectual properties (IPs) require substantial investment, including expert labor, large-scale data collection, cleaning and labeling, and significant computational resources for training. Despite this cost, DNN IPs remain vulnerable to Side-Channel Attacks (SCAs) [7], [8], [9], [10], [11], [12], [13], which can compromise sensitive model information such as architecture [8], [9], [12], dimension information [11], [12], input properties [14], and even model weights [15]. By exploiting side-channel leakages, these attacks pose serious threats to the security and privacy of DNN models.

A typical attack scenario is that a DNN is developed and trained locally, then deployed on third-party cloud or edge devices (e.g., in-vehicle embedded systems) for inference. This creates opportunities for attackers to exploit side-channel leakages, such as timing information, DRAM transactions, and cache patterns [8], [11], [16]. Attackers can collect such traces offline on the same type of device (e.g., an identical GPU) and use them to train deep learning-based predictors, such as LSTM [12] or attention-LSTM networks [17], to infer victim-model information (e.g., layer sequences) from runtime traces. With the recovered information, adversaries can build substitute models with similar architectures and comparable performance, or exploit the extracted information for downstream attacks [12], [18].

To mitigate SCAs and protect DNN IPs, many countermeasures have been proposed. Prior work explored memory-access obfuscation and hardware modifications to hide access patterns [19], [20], [21], but these methods often incur high memory overhead or are impractical for existing devices. Adversarial machine learning has also been used to defend against SCAs at the computer architecture layer [22], but such defenses require specific hardware deployments and are not portable across platforms. Apache Tensor Virtual Machine (TVM) [23], originally developed for efficient DNN inference across diverse architectures, hardware platforms, and deep learning frameworks, has also been suggested as a potential SCA countermeasure [12]. However, [16] showed that standard TVM optimizations are insufficient, and [17] further proposed an attack framework that can accurately recover DNN layer sequences after TVM optimization.

The work in [16] proposes a full-stack tool that obfuscates both high-level scripts and backend operations to protect DNN architectures. At the scripting level, function-preserving knobs (e.g., layer branching, deepening) modify architectures via high-level frameworks such as PyTorch. At the backend level, built

Received 2 November 2024; revised 8 April 2026; accepted 20 May 2026. Date of publication 27 May 2026; date of current version 1 September 2026. The work Yidan Sun and Siew-Kei Lam was supported in part by NTU-DESAY SV Research Program under Grant 2018-0980 and in part by the Ministry of Education, Singapore, under its Academic Research Fund Tier 2, under Grant MOE-T2EP20121-0008. The work of Guiyuan Jiang was supported by the National Natural Science Foundation of China under Grant 62372421 and Grant 62402465. *(Corresponding author: Guiyuan Jiang.)*

Yidan Sun is with the Imperial Global Singapore, Imperial College London, SW7 2AZ London, U.K. (e-mail: y.sun1@imperial.ac.uk).

Guiyuan Jiang and Peilan He are with the School of Computer Science and Technology, Ocean University of China, Qingdao 266100, China (e-mail: jiangguiyuan@ouc.edu.cn; hepeilan@ouc.edu.cn).

Jiayang Liu and Siew-Kei Lam are with the Cyber Security Research Centre @ NTU (CYSREN), the College of Computing and Data Science, Nanyang Technological University, Singapore 639798, ; (e-mail: jiayang.liu@ntu.edu.sg; siewkei_lam@pmail.ntu.edu.sg).

Qian Sun is with the Information Science and Technology, Nanjing University, Nanjing 300350, China (e-mail: sunqian@nuist.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TDSC.2026.3697449, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3697449

on TVM, selective fusion and schedule modifications obfuscate side-channel traces. A Genetic Algorithm (GA) leverages a known attack model (e.g., [12]) to search knob combinations meeting latency constraints. However, [16] does not assess efficiency, and its GA guidance by a single attack model limits robustness amid evolving SCA models.

DNN-Alias [24] extends the GA framework of [16] with an attack-agnostic evaluation: the GA minimizes per-feature standard deviations across GPU kernels. It restricts obfuscation to the script/architecture level (altering layer sequences) and still incurs long runtimes. The work in [25] studies architecture-level obfuscation via Neural Architecture Search (NAS). However, it assumes that the target DNN belongs to a NAS benchmark (e.g., NB-101, NB-303) and restricts the obfuscation knobs accordingly. In contrast, our setting does not assume a known NAS benchmark or precomputed accuracy labels, so the two settings are related but not directly comparable. [26] focuses solely on scheduling-level GPU obfuscation, offering efficient, attack-agnostic defense but omitting architecture-level obfuscation and lacking guarantees on latency constraints. [16] combines architecture- and backend-level obfuscation with latency awareness and reports state-of-the-art (SOTA) SCA protection. However, both [16], [24] scale poorly because they require time-intensive on-device deployment of candidate obfuscations for trace collection. Our experiments show that [16] takes over 120–150 hours to obfuscate only 20 DNNs, with cost increasing with layer count (Section IV-B). Moreover, its backend obfuscation requires expert-level TVM source modifications, limiting usability and robustness to TVM updates. Finally, evaluation with a single attack model raises concerns about generalization.

Despite recent progress, existing DNN obfuscation defenses against side-channel attacks (SCAs) still face fundamental limitations. Hardware- and memory-based methods are costly and device-specific, limiting deployment. Compiler-level optimizations (e.g., TVM) do not prevent advanced SCAs, introduce runtime overhead, and lack portability. Existing frameworks often rely on repeated device-side profiling and a single attack model, restrict the design space (e.g., NAS dependence), or fail to guarantee latency. These limitations call for a scalable, deployment-friendly obfuscation framework that is efficient and robust across platforms and attack models. In practice, obfuscation is rarely a one-shot process. Developers often need to evaluate multiple candidates under latency and functionality constraints, validate them on target hardware, and regenerate obfuscated models when the deployment setting or model changes. Therefore, generation time is a key factor affecting the scalability and usability of DNN protection in real deployment.

In this paper, we address the limitations of existing DNN obfuscation methods with a contrastive learning framework that improves robustness against SCAs at much lower cost, while enforcing latency constraints via backtracking. Our goal is to maximize obfuscation effectiveness against SCAs under a user-specified latency budget, rather than minimize the inference latency of the obfuscated model. We propose a unified framework that integrates architecture- and scheduling-level obfuscation. Two triplet networks treat the victim DNN as the anchor and rank candidates by their distance in the learned embedding space, so only top candidates are deployed for hardware evaluation, significantly reducing time cost. Scheduling-level obfuscation is implemented by modifying only TVM outputs (e.g., log files), making it more stable and easier to implement than [16]. Finally, we evaluate our method with two SOTA attack models to demonstrate robustness across attack models. Our contributions are as follows:

- We are the first to address the efficiency of DNN obfuscation by proposing CLOb-DNN, a contrastive learning framework that produces robust solutions at a fraction of the time required by SOTA methods. We reframe obfuscation selection as a metric-learning problem, where triplet loss enforces relative robustness rankings among candidates, directly matching the defender's decision objective. Two triplet networks, tailored for architecture-level and scheduling-level obfuscation, enable rapid filtering of effective solutions and substantially reduce obfuscation time.
- We designed sampling strategies for both architecture-level and scheduling-level obfuscation to support the triplet networks in learning an embedding space for selecting obfuscation solutions that are effective against multiple attack models. This involves creating negative-positive pairs, where negative samples exhibit stronger defense capabilities than positive ones.
- We conduct extensive experiments comparing our method with two SOTA approaches in terms of efficiency (time cost), (measured by ler), and latency, where the label error rate (ler) denotes the normalized edit distance between the attacker-recovered and ground-truth layer sequences. A higher ler reflects greater reconstruction mismatch and thus stronger protection. Our results demonstrate a 90% reduction in obfuscation time while attaining the highest average ler, surpassing SOTA baselines in defense capability. We further validate feasibility under diverse latency constraints, from strict (0.001 s) to relaxed (0.003 s), where CLOb-DNN consistently produces valid solutions, underscoring the importance of backtracking for meeting stringent budgets.
- Finally, we assess training efficiency and adversarial resilience: even when attackers have access to training and testing data, our obfuscated models impose higher replication costs and markedly reduce the effectiveness of white-box adversarial attacks.

II. Background

A. Threat Model of SCA on Optimized DNN Model

Optimization is widely used to reduce DNN inference latency. Apache TVM [23], [27], an open-source end-to-end compiler, automatically optimizes models at the graph and schedule levels to provide performance portability across CPUs, GPUs, and FPGAs, translating high-level scripts (e.g., PyTorch) into hardware-specific code. Although TVM has been explored as a countermeasure to side-channel attacks [12], [16], such optimizations can still be attacked: Sun et al. [17] proposed a two-phase framework that recovers optimized DNN layer

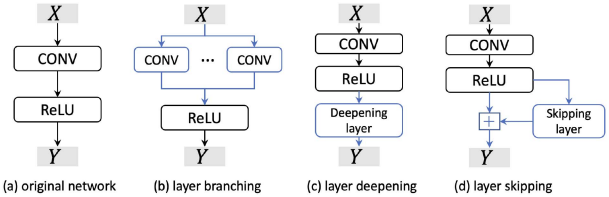


Fig. 1. Illustration of layer sequence obfuscation knobs in architecture-level.

sequences with high accuracy. Consistent with prior work [12], [16], [17], we consider an attack scenario where a locally trained victim DNN is optimized by tools such as TVM and deployed on untrusted third-party edge platforms (e.g., GPUs) for inference. The attacker has normal user-level privileges, can invoke inference, control inputs, and observe outputs. We consider the same side-channel features as prior studies: i) **Timing Information**: precise per-operator latency at each time step [8], [28], [29]; ii) **DRAM Access Patterns**: per-operator DRAM read/write information via PCIe side channels [12], [29]; iii) **Cache Behaviors**: L1/L2 cache transactions, utilization, and hit rates [29].

We study two SOTA attack models for stealing optimized DNN architectures: **DeepSniffer+** and **TPAM**. For fairness, both attacks are assumed to access the same side-channel features. i) **DeepSniffer+** [12] uses GPU kernel traces to train an LSTM with CTC loss for layer-sequence inference; [16] further improves it by omitting simple layers (e.g., BatchNorm, ReLU) to shorten label sequences. ii) **TPAM** [17] is a two-phase framework: Phase I learns mappings from noisy side-channel signals to operation distributions, and Phase II exploits temporal correlations between operations and layers to reconstruct the layer sequence.

Attacker's Goal, Capabilities, and Knowledge: The attacker seeks to reconstruct the optimized model architecture from side-channel traces collected during inference. They have user-level access on the same platform, can query the model and craft inputs (active), and can passively monitor timing, DRAM traffic, and cache contention observable in user space. They are aware that the model is compiled by a TVM-like optimizer and may leverage similar toolchains for training attack models, but they lack knowledge of the exact architecture, weights, or schedule parameters, and have neither privileged rights nor physical probing capability.

B. DNN Architecture Obfuscation

To defend DNNs against side-channel attacks, we adopt layer-sequence obfuscation following the SOTA method NeurObfuscator [16], which applies function-preserving transformations to alter the architecture without changing the model output. For a victim DNN M_i, the obfuscated architecture is denoted by a_i^k. As shown in Fig. 1, we consider three types of obfuscation: 1) **Layer branching**, which splits an original layer (e.g., *conv2d*) into multiple equal-size layers along the input or output dimension, with their outputs added or concatenated to preserve equivalence; 2) **Layer deepening**, which inserts an additional layer that leaves the original output unchanged, e.g., an identity layer for *fcn*; and 3) **Layer skipping**, which introduces skip-connected layers initialized with zero weights, so that the architecture is modified while the functional behavior is preserved.

C. DNN Scheduling Obfuscation

As shown in Fig. 2, for a given DNN, TVM's backend (e.g., "relay," "AutoTVM") extracts tasks from high-level scripts (e.g., PyTorch, TensorFlow). Tasks are characterized by type (e.g., "conv2d," "dense") and workload details (e.g., input/output shapes, kernel size). TVM then assigns and tunes configurations per task to optimize performance. In Fig. 2, a simple 2D convolution in PyTorch takes an input in $\mathbb{R}^{3\times32\times32}$ (input channel, height, and width of the image) and applies a kernel in $\mathbb{R}^{128\times3\times3\times3}$ (output channel, input channel, and kernel dimensions). After extraction as a task (① "relay" + "AutoTVM"), the workload tensors are [1,3,32,32] (input) and [128,3,3,3] (kernel). We denote the input channel (3) as $tile\_x$ and the output channel (128) as $tile\_y$ (other dimensions appear in Section III-C). For this standard conv2d, TVM provides code templates (right of Fig. 2). Each dimension is split into loops; for example, $tile\_y$ is partitioned into four loops, $tile\_y.0$–$tile\_y.3$, subject to $tile\_y.0 \times tile\_y.1 \times tile\_y.2 \times tile\_y.3 = 128$. The key challenge is choosing parameter values that satisfy such constraints while optimizing performance.

TVM aims to identify the optimal combination of parameters $tile\_y.0$ to $tile\_y.3$ that lead to the shortest DNN latency while satisfying given constraints. During the tuning phase (i.e., ② "XGBTuner.tune"), numerous configurations are explored and evaluated, as illustrated in the center of Fig. 2. For example, $tile_y$ might be set to $tile\_y.0 = 2$, $tile\_y.1 = 8$, $tile\_y.2 = 2$, and $tile\_y.3 = 4$, respectively. AutoTVM samples many such configurations (e.g., config i, j, k in Fig. 2), evaluates their latency using a tuner like XGBTuner, records all results, and eventually applies the best configuration (i.e., ③ "apply_history_best"). For a DNN M_i, we denote the corresponding TVM-generated optimal configuration file as c_i^{best}; Moreover, for an obfuscated architecture based on M_i, i.e., a_i^k, the configuration file is $c_{i,k}^{best}$. In the context of the SCA problem, the objective shifts from latency minimization to maximizing security level, i.e., the ler performance over considered attack models, which introduce new challenges.

Defender's Goal, Capabilities, and Knowledge: The defender seeks to maintain model functionality and accuracy, while obfuscating architecture and scheduling configurations to mitigate SCAs. They employ function-preserving obfuscations (such as branching, deepening, and skipping) to generate alternative architectures, and control TVM compilation to obtain optimized schedules for each variant. Their knowledge encompasses the full model and TVM workload structure (e.g., operator types, tensor shapes, tiling and scheduling parameters), as well as latency–security trade-offs. Within practical latency budgets, they assume the ability to explore multiple schedule configurations, reframing compilation from latency minimization to maximizing security (higher LER) without altering model outputs.

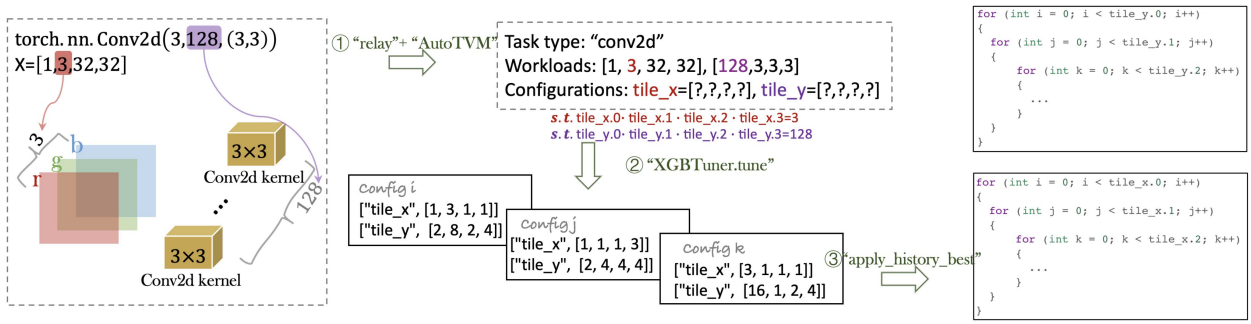


Fig. 2. Illustration of how TVM tunes *scheduling-level configurations* for a simple 2D convolution layer. The input image has a shape of $3 \times 32 \times 32$, and the convolution kernel, shaped $128 \times 3 \times 3 \times 3$, performs convolution over the image. The input channel has a size of 3, while the output channel is of size 128. The tuning process involves the following steps: ① TVM extracts the layer from PyTorch script as a task where task type and workload will be identified; ② It explores and tunes various configurations for specific dimensions to optimize performance. ③ Finally, the device executes the code with the best configuration.

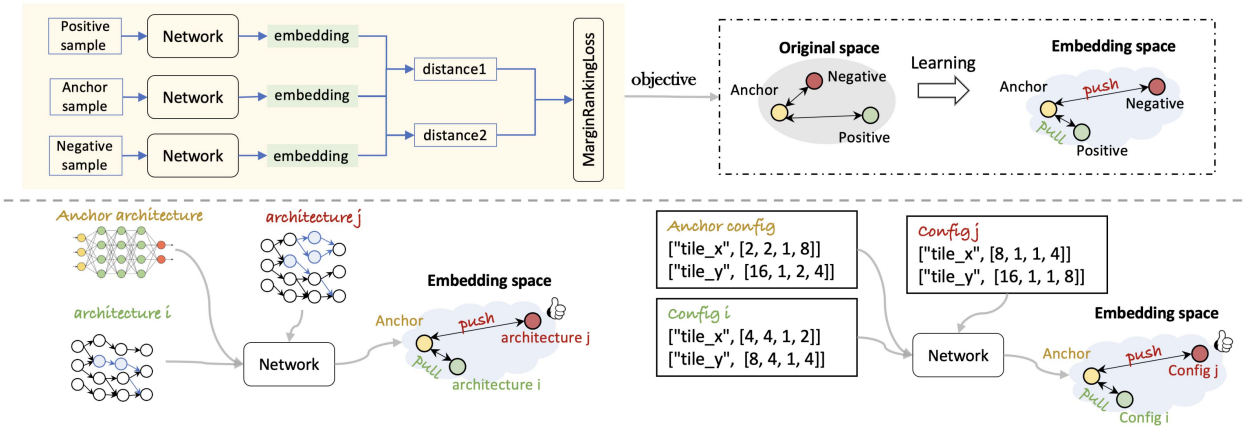


Fig. 3. Utilizing triplet network for fast obfuscation.

III. Our Method

A. Overview Framework

This section presents our contrastive learning framework, *CLOb-DNN:* As shown in the upper half of Fig. 3, a triplet network contains three identical subnetworks with shared parameters, mapping anchor, positive, and negative samples into a learned embedding space. In person identification, the anchor and positive are images of the same person, while the negative is from a different person. Trained with *MarginRankingLoss*, the network reduces the anchor-positive distance relative to the anchor-negative distance, clustering similar samples and separating dissimilar ones. After training, classification is performed by comparing a sample's distance to a known anchor with a threshold. Triplet loss optimizes relative orderings rather than class labels, which matches our goal: among feasible candidates, select those that are harder to attack. By defining negatives as more robust than positives, the learned margin induces a monotonic ranking with respect to measured robustness, yielding an attack-agnostic surrogate aggregated over multiple SCA models.

Based on this idea, we train triplet networks to efficiently select obfuscations for a new victim DNN from a candidate set. As shown below the dashed line in Fig. 3, the victim DNN serves as the anchor, representing its original architecture and configurations. We project N_{arc} architecture-level candidates into the learned embedding space and prefer those farther from the anchor, since they are more distinct in the side-channel feature space. This enables one-shot inference-based selection without extensive deployment and profiling, making it much faster than SOTA methods [16]. Similarly, scheduling-level configurations are selected by another triplet network (bottom-right of Fig. 3). We next describe the two triplet networks.

The core principle of our framework is that the learned embedding space provides an efficient proxy for defense effectiveness (ler). This relationship is causal: obfuscation alters a DNN's side-channel signatures, and our contrastive learning approach explicitly trains the network to map the semantic dissimilarity of these signatures to geometric distance. A greater distance from the anchor, therefore, serves as a principled measure of the signature disruption that causes a higher ler. A detailed theoretical grounding for this approach is provided in *Supplementary Material*.

B. Selecting Architecture Obfuscation Solutions

This section presents our architecture obfuscation framework (Fig. 4), including 1) *Dataset Construction*, 2) *Offline Training*, and 3) *Online Testing* for selecting the optimal obfuscation under

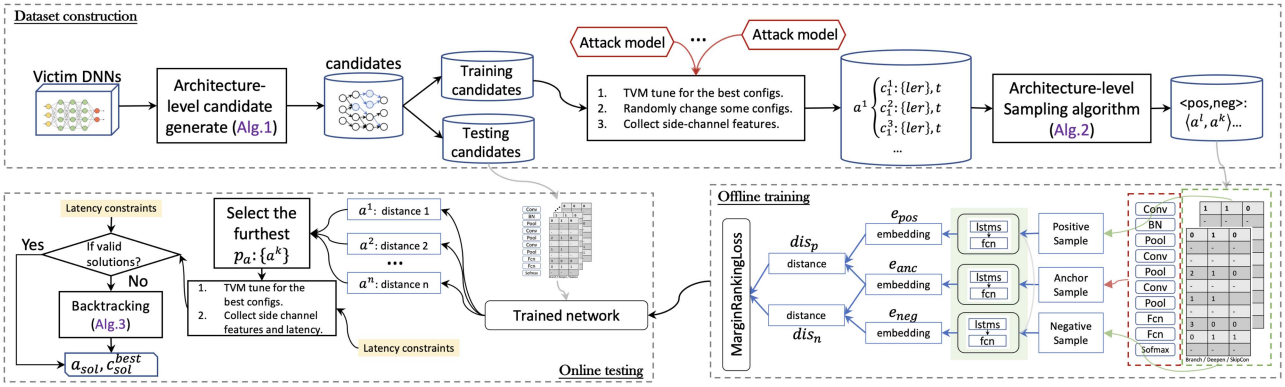


Fig. 4. Overview framework of our architecture-level obfuscation, which includes offline-training and online-testing.

TABLE I
NOTATIONS

Notation	Description
M_i	DNN model i.
$\mathbf{la}_i$	the layer sequence of M_i.
H_{M_i}	the historical recorded scheduling configurations during TVM optimization process.
a_i^k	the k-th obfuscated architecture of M_i.
c_i^{best}	the TVM optimized best configuration file of original M_i.
$c_{i,k}^{best}$	the TVM optimized best configuration file of obfuscated architecture a_i^k of M_i.
c_i^j	the j-th obfuscated configuration file of original architecture M_i.
$c_{i,k}^j$	the j-th obfuscated configuration file of architecture a_i^k of M_i.
N_{arc}	the number of expected architecture candidates.
K_{arc}	the set of obfuscation knobs (layer branching/deepening/skipping).
$Obf_{arc}^i = \{a_i^k\}$	the set of obfuscated architecture samples of mode M_i.
$S_{arc}^i = \{< a_i^l, a_i^k >\}$	the set of obfuscated architecture positive and negative sample pairs of mode M_i.
$\hat{a}_i^k$	a back-tracked architecture based on a_i^k within latency constraint.
N_{sch}	the number of expected configuration candidates.
$Obf_{sche}^{i,k} = \{c_{i,k}^j\}$	the set of obfuscated configuration samples of mode a_i^k.
$S_{sche}^{i,k} = \{< c_{i,k}^j, c_{i,k}^o >\}$	the set of obfuscated configuration positive and negative sample pairs of obfuscated architecture a_i^k of mode M_i.
n_{arc}/n_{sch}	the size of final embedding vector of architecture-/scheduling-level triplet network.
p_a/p_c	the number of the top furthest architectures/ configurations during online testing pipeline.

a time budget. We assume the victim DNN has been TVM-optimized and that its optimal and auxiliary configuration files are available. Our goal is to obfuscate the target using only this information.

1) Architecture-Level Sample Generation: Unlike computer vision tasks, where inputs are easily distinguishable images, our task requires detailed analysis and careful dataset design. This involves handling complex inputs, like obfuscated architectures and configuration files, and defining positive and negative samples based on security levels across various attack models and latency considerations.

Unlike computer vision tasks, where inputs are easily distinguishable images, our task requires careful dataset design due to complex inputs, such as obfuscated architectures and configuration files, and the need to define positive and negative samples by security level across attack models under latency constraints. To address this, we design a two-algorithm pipeline, shown in the upper box of Fig. 4. First, each victim DNN is processed by Algorithm 1 to generate N_{arc} (e.g., 100) obfuscated architecture candidates. A victim DNN M_i consists of a layer sequence $\mathbf{la}_i$ (right box of Fig. 4); its candidate set Obf_{arc}^i contains the corresponding obfuscation matrices. Algorithm 1 excludes layers unsuitable for obfuscation, such as batch normalization and pooling, and randomly selects layers for branching, deepening, and skip connections. For branching, input-dimension options $\{1, 2\}$ split a layer into 2 or 4 equal-sized layers; output-dimension options $\{3, 4\}$ yield the same effect. For deepening and skip connections, the option is binary, indicating whether the transformation is applied. Each candidate is represented as a matrix $a_i^k \in \mathbf{R}^{|\mathbf{la}_i| \times 3}$, where rows correspond to layers and columns to obfuscation knobs; a zero entry means the corresponding knob is not applied to that layer. The full dataset Obf_{arc} is then split into training and testing sets: 80 DNNs $\times N_{arc}$ candidates for training, and 20 DNNs $\times N_{arc}$ candidates for testing.

Defining positive and negative samples accurately before training is crucial. For a given architecture, whether original M_i or obfuscated (e.g. a_i^k), multiple configuration files can influence the side-channel feature distribution (e.g., c_i^{best} or $c_{i,k}^{\text{best}}$ from TVM). We observed that some architectures consistently show higher ler across various configurations. Fig. 5 shows results for three baseline DNNs. In each subfigure, DNN refers to the fixed original architecture (without obfuscation). The x-axis lists 30 architecture-level obfuscated candidates derived from that baseline, and the y-axis reports their average ler. Colored curves correspond to distinct scheduling configurations (cfg), each evaluated at different scheduling-level obfuscation ratios (20%, 40%, 60%, 80%, 100%). Most architectures that perform well under one configuration also perform well under others (black-box), though exceptions occur (red box). We observe obfuscated architectures that are both effective and robust across configurations, enabling a reduced search space by prioritizing them. The central challenge is identifying these robust architectures within the candidate set.

Algorithm 1: Architecture-Level Candidate Generation.

Input: Layer sequence $\mathbf{la}_i$ of model dnn M_i, pool of architecture obfuscation knobs K_{arc}, sample number N_{arc}.
Output: Candidate set: Obf^i_{arc}.

Identify layers $\mathbf{la}_i^{'}$ could be applied obfuscation knobs (e.g., exclude pooling/batch norm layers).
while $|Obf^i_{arc}| < N_{arc}$ **do**
  for *knob in* K_{arc} **do**
    Randomly select a number n_o in range $[1, len(\mathbf{la}_i^{'})]$.
    Randomly select n_o locations layers in $\mathbf{la}_i^{'}$.
    for *each location* **do**
      Randomly select a optional obfuscation value to obfuscate.
      ▷ e.g., if knob==branching, optional could be $\{1, 2, 3, 4\}$
    end
  end
  Add new architecture into Obf^i_{arc} if it does not already exist in Obf^i_{arc}.
end

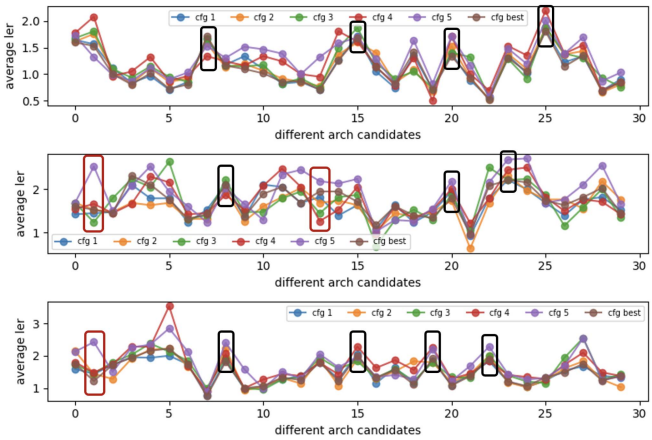


Fig. 5. *ler* (Label Error Rate) quantifies the discrepancy between attacker-recovered and true layer sequences (higher *ler* indicates stronger defense).

Therefore, we propose the three-step process in Fig. 4: 1) *Optimal Configuration Identification:* for each architecture candidate a_i^k, TVM finds the optimal configuration file $c_{i,k}^{\text{best}}$, since obfuscation introduces new layers requiring new configurations. 2) *Diverse Configuration Generation:* for each a_i^k, we generate four additional configurations, $c_{i,k}^1, c_{i,k}^2, c_{i,k}^3, c_{i,k}^4$, by modifying 20%, 40%, 60%, and 80% of the layers. 3) *Side-Channel Feature Collection:* we collect side-channel features on the target device for all candidates under each configuration; attack models infer layer sequences from the traces and produce latency and *ler*, which are then used by Algorithm 2 to identify negative (expected) and positive (unexpected) samples. In summary, this process counts how many candidates each a_i^k dominates across configurations; a higher count indicates greater robustness and helps assess high-security candidates under diverse configurations.

Specifically, for each candidate architecture $a_i^{k_1} \in Obf^i_{arc}$ under a given configuration (e.g., the 20% altered configuration c^2_{i,k_1}), we identify the architectures it dominates.

Algorihtm 2: Architecture-Level Positive-Negative Pairs Sampling Strategy.

Input: N_{arc} obfuscated architecture candidates Obf^i_{arc} for model M_i, each architecture a_i^k is associated with some changed configurations $\{c^j_{i,k}\}$, and corresponding $\{ler\}$s.
Output: Sampling pairs set for M_i: S^i_{arc}.

$Num_{dominate} = \{\}$
for *each candidate* $a_i^k \in Obf^i_{arc}$ **do**
  For each specific configuration file, obtain corresponding set of candidates dominated by a_i^k.
  For above all sets, obtain their intersection set ss.
  $Num_{dominate}[a_i^k] = |ss|$
end
Split Obf^i_{arc} into two disjoint sets: Obf_A and Obf_B, where $a_i^k \in Obf_A$ satisfies that to improve the ler based on all $\{c^j_{i,k}\}$ over all attack models, otherwise, put into Obf_B.
Sort samples in Obf_A and Obf_B based on the $Num_{dominate}$ in descending order.
for k *in range*(a_{num}) **do**
  a_i^k is the first k-th sample, as the negative sample.
  Randomly select b_{num} different positive samples $\{a_i^l\}$ from Obf_A, which are behind a_i^k.
  for $a_i^l \in \{a_i^l\}$ **do**
    Add sample pairs $< a_i^l, a_i^k >$ into S^i_{arc}.
  end
end
Randomly select c_{num} different pairs $< a_i^l, a_i^k >$ add into S^i_{arc}, where $a_i^k \in Obf_A$ and $a_i^l \in Obf_B$.

We say that $a_i^{k_1}$ under c^2_{i,k_1} dominates $a_i^{k_2}$ under c^2_{i,k_2} if its *ler* is consistently higher for every attack model. We then compute the intersection of these dominance sets across all configurations, including the optimal and altered ones, and define its size as $Num_{dominate}[a_i^k]$. Thus, $Num_{dominate}[a_i^k]$ measures how many candidates are consistently dominated by a_i^k across configurations; a larger value indicates greater robustness and captures the stability of high-security candidates under different scheduling configurations.

Next, as shown in Algorithm 2, we divide Obf^i_{arc} into two subsets, Obf_A and Obf_B. Obf_A contains candidates that improve *ler* for all attack models under all configurations. We sort both subsets by $Num_{dominate}[a_i^k]$. From Obf_A, we select the top a_{num} candidates as negative samples, representing the most robust architectures, and randomly choose b_{num} candidates from the remaining Obf_A as positive samples, creating a clear margin between the strongest and next-best candidates. Specifically, we form triplets (anchor, positive, negative), where the anchor is the original model, the positive is a weaker candidate, and the negative is a stronger one. Concretely, candidates in Obf_A are treated as negatives and those in Obf_B as positives, so stronger candidates are pushed farther from the anchor than weaker ones. This sampling uses dominance counts and defense performance across all attack models to identify robust, high-security architectures, on which we train the triplet network to separate strong (negative) from weak (positive) candidates.

2) Training and Testing of Architecture-Level Triplet Network: In our approach (illustrated in the bottom right of Fig. 4), we treat the victim DNN as an anchor sample. Its layer sequence

(e.g., "conv"-"BN"-"Pool"-$\cdots$) is encoded into a sequence of integers, which is input into an anchor network comprising three Long Short-Term Memory (LSTM) layers and a Fully Connected (FC) layer. This network outputs a latent matrix of dimensions $\mathbf{R}^{|la|\times n_{arc}}$, where $|la|$ is the number of layers and n_{arc} is the embedding vector size. The LSTM layers capture temporal correlations between layers, and the FC layer projects the output into an embedding space. To obtain a fixed-size embedding vector $\mathbf{e}_{anc} \in \mathbf{R}^{n_{arc}}$, we sum the latent matrix along the first dimension, accommodating varying $|la|$ across different victim DNNs.

Obfuscated architectures are represented by an obfuscation matrix $a_i^k \in \mathbf{R}^{|la|\times 3}$, whose entries are zero (no obfuscation) or nonzero integers (obfuscation). They are processed by a network with the same structure as the anchor network but larger LSTM layers to capture complex correlations between layers and obfuscation knobs; both positive and negative samples are fed into this network. Although Fig. 4 shows two networks with shared architecture and parameters, a single network is used in practice during training and testing. During training, the network outputs embedding vectors $\mathbf{e}_{pos}$ and $\mathbf{e}_{neg}$, with the same dimension as $\mathbf{e}_{anc}$. We then compute the *L2*-norm distances between $\mathbf{e}_{anc}$ and $\mathbf{e}_{pos}$, denoted by dis_p, and between $\mathbf{e}_{anc}$ and $\mathbf{e}_{neg}$, denoted by dis_n. To enforce $dis_p < dis_n$, we use the Margin Ranking Loss:

$$loss(x1, x2, y) = \max(0, -y(x1 - x2) + margin) \quad (1)$$

where $y = -1$, $x1 = dis_p$, $x2 = dis_n$, and $margin = 1.0$.

After training, for a new victim DNN we assume access only to its architecture and TVM-generated configurations. Using Algorithm 1, we produce N_{arc} (e.g., 100) obfuscated candidates and feed them to the triplet network, which computes their distances to the anchor in the embedding space. We then sort and select the top p_a (e.g., 5) most distant candidates within seconds. Because this selection is security-only and ignores performance (e.g., latency), the top p_a are not adopted directly. Instead, we deploy them on the target device, tune with TVM, and collect side-channel features and latency. Attack models evaluate the ler distribution; among candidates satisfying the latency constraint, we choose the one with the highest average ler as the final solution.

When all p_a candidate solutions fail to satisfy strict constraints (e.g., latency), we use a backtracking algorithm (Algorithm 3) to trade obfuscation strength for lower latency by reverting selected obfuscated layers to their original states. Given an obfuscated architecture a_i^k with latency $t_{a_i^k}$ and budget t_{budget}, the algorithm computes N_{knob}, the number of layers to backtrack, based on the total number of obfuscation knobs, the gap between $t_{a_i^k}$ and t_{budget}, and a coefficient c (e.g., 0.9) that reserves capacity for scheduling obfuscation. In each iteration, the algorithm randomly generates five backtracked architectures by modifying N_{knob} layers and deploys them on the target device. If any satisfies the latency constraint, the one with the highest ler is selected and the algorithm terminates. Otherwise, N_{knob} is increased by a step size (e.g., step $= 3$). This procedure enforces performance constraints while preserving as much security as possible.

Algorithm 3: `Architecture Back-Tracking.`

Input: Obfuscated architecture a_i^k, latency of a_i^k: $t_{a_i^k}$, latency constraint t_{budget}.
Output: Back-tracked architecture $\hat{a}_i^k$.

N_{knob} is total number of knobs which are applied to the DNN in a_i^k.
$N_{back} = \max(3, int(N_{knob} * \frac{t_{a_i^k} - c * t_{budget}}{t_{a_i^k}}))$.
while $t_{\hat{a}_i^k} > c * t_{budget}$ **do**
  for *i in range(5)* **do**
    Randomly select N_{back} obfuscated layers in a_i^k back to no obfuscation as $\hat{a}_i^k$.
    Deploy $\hat{a}_i^k$ on the target device collect latency and ler.
  end
  Find the $\hat{a}_i^k$ with the highest ler while satisfying $t_{\hat{a}_i^k} <= c * t_{budget}$.
  if *find valid* $\hat{a}_i^k$ **then**
    return $\hat{a}_i^k$.
  end
  else
    $N_{back} += step$
  end
end

C. Efficient Scheduling Configuration Selection

After obtaining an obfuscated architecture a_{sol} and its optimal configuration c_{sol}^{best} that satisfy security and performance targets, we perform scheduling-level obfuscation. As in the architecture stage, we train a triplet network to learn a ranking-oriented embedding of configurations, reducing the search space and accelerating solution generation. Although the pipeline is similar to Fig. 4 (see Fig. 6), this section highlights scheduling-specific differences.

Before describing each module, we first explain the format and properties of a "configuration file," which reveal the challenges of scheduling-level obfuscation. As shown in Fig. 7, for a simple CNN (left), the TVM-generated configuration file contains three components: task type, workload tensors, and configurations. In this example, the CNN is decomposed into configurable tasks such as "conv2d" and dense layers, excluding pooling and batch normalization layers because they have no tunable parameters. Workload tensors specify task dimensions. For example, [1,3,32,32] denotes batch size, input channels, and input height/width, while [32,3,3,3] denotes output channels, input channels, and kernel height/width. These tensors directly determine the configuration values. For instance, the input channel 3 (orange) corresponds to `tile_rc`, which is split into two loops satisfying `tile_rc.1*tile_rc.2`$= 3$ (e.g., [1,3]); similarly, the output channel 32 (purple) corresponds to `tile_f`, split into four loops [16,1,2,1] satisfying $16 * 1 * 2 * 1 = 32$. In summary, each configurable task is defined by its task type, workload tensors, and associated configurations.

1) Scheduling-Level Obfuscation Sample Generation: To generate candidate scheduling-level obfuscations for a TVM-optimized victim DNN, we use the best configuration file and historical records as prior knowledge, as shown in the

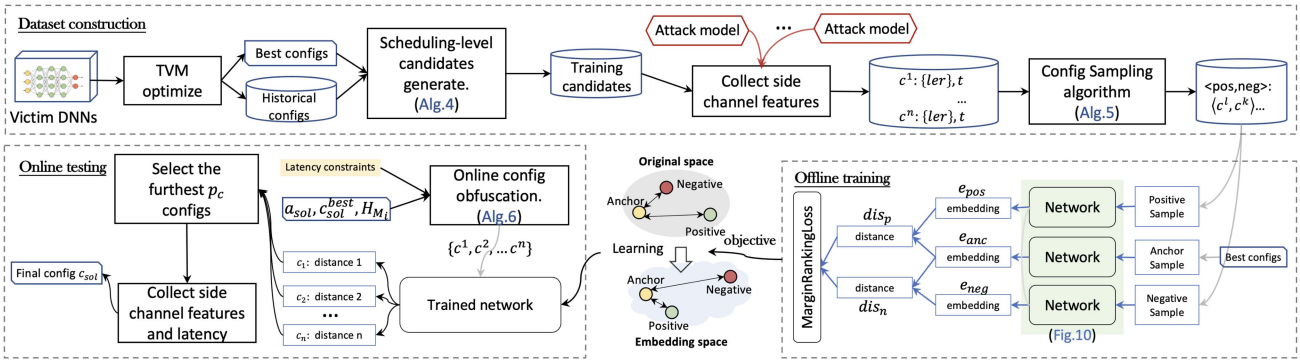


Fig. 6. Overview of our scheduling-level obfuscation framework, illustrating both offline training and online testing phases.

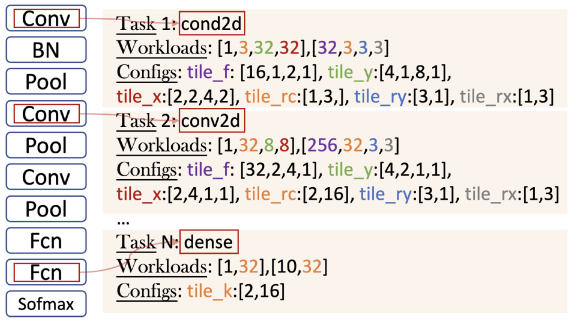


Fig. 7. Detailed configurations of a given toy CNN.

upper part of Fig. 6. For example, a configurable dimension such as `tile_y` (middle of Fig. 2) can satisfy its constraints through multiple historical settings, e.g., [2,8,2,4], [2,4,4,4], and [16,1,2,4]. TVM evaluates such configurations by on-device measurement or latency estimation and selects the best one. Following Algorithm 4, we first identify configurable tasks in the final TVM-generated configuration file. For each task, we randomly select an alternative configuration from its historical record set H_{M_i} and assemble them into a new configuration file c_{obf}. If $c_{\text{obf}} \notin Obf_{\text{sche}}$, it is added as a new candidate. This process repeats until $|Obf_{\text{sche}}| = N_{\text{sch}}$.

From the 80 training DNNs, we generate $80 \times N_{\text{sch}}$ candidate configurations. To evaluate ler under different attack models, each configuration is deployed on the target device. Following Algorithm 5, we partition Obf^i_{sche} into Obf_A and Obf_B, where Obf_A contains candidates that improve ler for all attack models. Unlike the architecture-level case, we rank Obf_A and Obf_B directly by average ler across attack models. From Obf_A, we select the top a_{num} candidates as negative samples and randomly choose b_{num} from the remainder as positive samples, enlarging the gap between the strongest and next-best candidates. We further sample negatives from Obf_A and positives from Obf_B, so stronger candidates lie farther from the anchor than weaker ones. In this way, defense strength against multiple attack models is encoded as ordering information for triplet-network training.

2) Scheduling-Level Triplet Network: Unlike architecture-level obfuscation, the scheduling-level triplet network takes

Algorithm 4: Scheduling-Level Candidates Generation.

Input: The best scheduling configuration file c^i_{best} for model M_i, and corresponding historical records H_{M_i} (generated by TVM optimization process), expected candidate number N_{sch}.
Output: Candidate set: Obf^i_{sche}.

Extract tasks and configurations from c^i_{best}.
while $|Obf^i_{sche}| < N_{sch}$ **do**
  $c^k_i = null$
  for *each task* **do**
    Identify records $\{r_j\}$ in H_{M_i} belong to this task.
    Randomly choose one configuration record from $\{r_j\}$ which does not belong to c^i_{best} and add to c^k_i.
  end
  if $c^k_i \notin Obf^i_{sche}$ **then**
    Add c^k_i in Obf^i_{sche}.
  end
end

Algorithm 5: Scheduling-Level Positive-Negative Pairs Sampling Strategy.

Input: N_{sch} scheduling-level obfuscated configurations Obf^i_{sche} for model M_i, and corresponding $lers$.
Output: Sampling pairs set: S^i_{sche}.

Split Obf^i_{sch} into two disjoint sets: Obf_A and Obf_B, where $c^k_i \in Obf_A$ satisfies that to improve the ler over all attack models, otherwise, put into Obf_B.
Calculate average ler of each sample in Obf_A and Obf_B of over all considering attack models, respectively.
Sort samples in Obf_A and Obf_B based on the average ler in descending order.
for *k in range*(a_{num}) **do**
  c^k_i is the first k-th sample, as the negative sample.
  Randomly select b_{num} different positive samples $\{c^l_i\}$ from Obf_A, which are behind c^k_i.
  for $c^l_i \in \{c^l_i\}$ **do**
    Add sample pairs $< c^l_i, c^k_i >$ into S^i_{sche}.
  end
end
Randomly select c_{num} different pairs $< c^l_i, c^k_i >$ where $c^k_i \in Obf_A$ and $c^l_i \in Obf_B$.

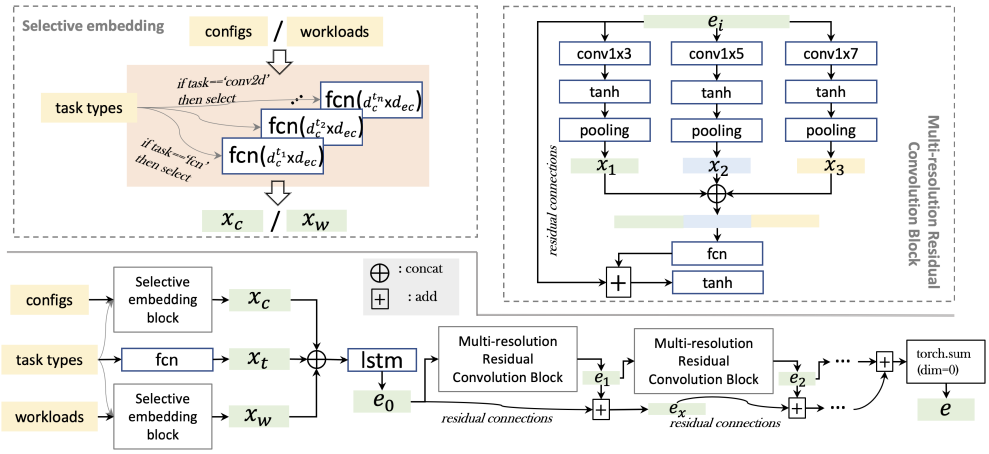


Fig. 8. Details of our scheduling-level multi-resolution convolution triplet network.

heterogeneous inputs: task types, workloads, and configurations. As shown in Fig. 7, workloads depend on task type; for example, "conv2d" has two length-4 tensors, while "dense" has two length-2 tensors. Moreover, different workload positions correspond to different configuration tensors: `tile_f`, `tile_y`, and `tile_x` use four parameters, whereas `tile_rc`, `tile_rx`, and `tile_ry` use two. This heterogeneity makes network design challenging, as diverse configuration files must be mapped into a unified embedding that still separates strong defensive samples. In addition, the victim DNN may contain an unknown number of layers, while the network must output a fixed-format embedding. Each task type has different workload sizes that define configuration boundaries. Even configurations satisfying the same constraint, e.g., [2,8,2,4], [2,4,4,4], and [16,1,2,4] with product 128, can produce different side-channel feature distributions at runtime.

To address these challenges, we design a triplet network (Fig. 8) with a *Selective Embedding* block and a recurrent network with *Multi-resolution Residual Convolution* blocks, mapping heterogeneous scheduling inputs into a unified embedding space. The *Selective Embedding* block contains two groups of fully connected networks (FCNs), one for workload tensors and the other for configuration tensors. Each group includes task-specific FCNs with input dimensions determined by the corresponding tensor length, but all produce fixed-dimensional outputs. This design allows the network to learn task-specific representations while aligning them in a shared feature space. For example, for "conv2d", the configuration FCN takes a 20-dimensional input ($d_c^{t_n}$) and outputs d_{ec}, while the workload FCN takes an 8-dimensional input ($d_l^{t_n}$) and outputs d_{el}. For "dense", the workload FCN takes a 4-dimensional input ($d_l^{t_1}$) and the configuration FCN takes a 2-dimensional input ($d_c^{t_1}$), while their output dimensions remain d_{el} and d_{ec}, respectively.

To provide an overview of the process, consider a victim DNN M_i with a configuration file c_k consisting of N_{tk} tasks (note that N_{tk} may vary among different DNNs). Each task record includes the task type, workload, and configurations. First, the task type is encoded as an integer (e.g., "fcn" is 1, "depthwise_conv2d" is 2,..., "conv2d" is n). This integer is input into a FCN with an input dimension of 1 and an output dimension of d_{et}. For all tasks in M_i, the output vectors are concatenated to form a matrix $x_t \in \mathbb{R}^{N_{tk} \times d_{et}}$. Similarly, within the "Selective Embedding" block, each task's workload tensor and configuration tensor are processed by their respective FCNs, and the resulting output vectors are concatenated to form matrices $x_w \in \mathbb{R}^{N_{tk} \times d_{el}}$ and $x_c \in \mathbb{R}^{N_{tk} \times d_{ec}}$, respectively. After the "Selective Embedding" block, the matrices x_t, x_c, and x_w are concatenated along the second dimension to obtain a matrix in $\mathbb{R}^{N_{tk} \times (d_{et}+d_{ec}+d_{el})}$. At this point, our selective mechanism has independently learned representations for different task types while aligning these diverse types into a unified feature space.

Considering that DNN execution follows a sequential layer order, we use a three-layer LSTM to capture temporal information. Its input dimension is $d_{et} + d_{ec} + d_{el}$, producing an embedding matrix $e_0 \in \mathbb{R}^{N_{tk} \times d_{h_0}}$, which also aligns task type, workload, and configuration in a unified feature space. To learn multi-level embeddings, we design a "*Multi-resolution Residual Convolution*" (MRRC) block that takes $e_i \in \mathbb{R}^{N_{tk} \times d_{h_i}}$ as input. As shown in the upper-right box of Fig. 8, three one-dimensional convolutional layers with kernel sizes 1×3, 1×5, and 1×7 are applied over the feature dimension of e_i. Each is followed by a tanh activation and pooling layer, producing three multi-resolution latent matrices (x_1, x_2, and x_3). These are concatenated along the feature dimension and passed through a FCN. A residual connection then adds the result to e_i, followed by another tanh activation, to mitigate vanishing and exploding gradients. We stack multiple MRRC blocks with residual connections between adjacent blocks to learn high-dimensional multi-resolution features and improve convergence. Finally, the embedding matrix is summed over the second dimension to obtain the final embedding vector $e \in \mathbb{R}^{d_{fe}}$, which represents a configuration file with variable task types and counts in a unified embedding space.

During training, as in the architecture-level case, the network in Fig. 8 is applied to the anchor configuration c_{best}, while positive and negative samples (c_l and c_k) are processed by networks of the same architecture. As shown in Fig. 6, this yields embedding vectors $\mathbf{e}_{\text{pos}}$ and $\mathbf{e}_{\text{neg}}$, both with the same dimension as the anchor vector $\mathbf{e}_{anc}$. We then compute the L2-norm

Algorithm 6: Online Obfuscated Configuration Generation.

Input: Obfuscated architecture a_{sol} of original victim DNN M_i, corresponding best configuration file c_{sol}^{best} and historical records H_{M_i}; latency constraint t_{budget} and the latency of a_{sol} under c_{sol}^{best}:t.
Output: Configuration candidate set: C_{obf}.

$\{tk^j\}$ are tasks extracted from c_{sol}^{best}.
$N_{obf} = |\{tk^j\}| * (\frac{t_{budget}-t}{t_{budget}}, 1)$.
$C_{obf} = null$.
while $|C_{obf}| < 50$ **do**
  Create a new empty configuration file c_{tmp}. Randomly select N_{obf} tasks from $\{tk^j\}$ as set A. **for** *each task* tk_j *in* A **do**
    Scanning H_{M_i} and extracted records with same task type and workload with tk_j.
    Randomly select one record which not in c_{sol}^{best} and add into file c_{tmp}.
  end
  if $c_{tmp} \notin C_{obf}$ **then**
    Add c_{tmp} into C_{obf}.
  end
end
return C_{obf}.

distances dis_n between $\mathbf{e}_{anc}$ and $\mathbf{e}_{\text{neg}}$, and dis_p between $\mathbf{e}_{anc}$ and $\mathbf{e}_{\text{pos}}$. The training uses the Margin Ranking Loss in (1) to enforce a larger margin, i.e., $dis_n > dis_p$.

3) Online Scheduling-Level Obfuscation: The lower-left box of Fig. 6 presents the pipeline of our online scheduling-level obfuscation, which aims to generate efficient and robust obfuscated configuration files. Given a new victim DNN model M_i that has already undergone architecture-level obfuscation, we have the obfuscated architecture a_{sol}, optimized by TVM to produce the best configuration file $c_{\text{sol}}^{\text{best}}$, along with historical configuration records H_{M_i} generated during the TVM optimization process.

Assuming a latency constraint t_{budget} and that the execution latency of a_{sol} under configuration $c_{\text{sol}}^{\text{best}}$ is less than t_{budget}, there is available latency margin for scheduling obfuscation. We propose Algorithm 6 to generate obfuscated configuration candidates while respecting the latency constraint. As detailed in Algorithm 6, we extract a set of configurable tasks $\{tk^j\}$ from the anchor configuration file $c_{\text{sol}}^{\text{best}}$, where all configurations have been tuned by TVM for minimal execution latency. To determine the number of tasks to obfuscate, we calculate: $N_{obf} = |\{tk^j\}| * \frac{t_{budget}-t}{t_{budget}}$, where $|\{tk^j\}|$ represents the total number of tasks in the obfuscated DNN a_{sol}, and $\frac{t_{\text{budget}}-t}{t_{\text{budget}}}$ is a ratio indicating the proportion of tasks to obfuscate. This ratio guides the algorithm to generate candidates that comply with the latency constraint. Algorithm 6 then randomly modifies N_{obf} task records by replacing the original configurations in $c_{\text{sol}}^{\text{best}}$ with random configurations from H_{M_i} that have the same task types and workloads but different configurations. By repeating this process 50 times, we generate 50 distinct candidates within seconds.

The anchor configuration c_{sol}^{best}, together with the candidates $C_{obf} = \{c^1, c^2, \ldots, c^n\}$ $(n = 50)$, are fed into the trained triplet network (Fig. 8) to compute their distances from the anchor in the embedding space. The p_c configurations farthest from the anchor are then evaluated on the target device to measure actual latency and collect side-channel features. The configuration achieving the highest average ler while satisfying latency constraint is selected as final solution c_{sol}.

In summary, our framework selects obfuscations in three stages: i) candidate generation via function-preserving architectural transformations and TVM schedule remapping; ii) robustness ranking via two triplet networks that provide an attack-agnostic surrogate metric and retain only top candidates for hardware evaluation; and iii) budget enforcement via backtracking when latency constraints are violated. Built on rigorous preliminary studies and observations, each step is designed to jointly address security against multiple attack models, latency constraints, and obfuscation efficiency.

IV. Experiment and Discussion

A. Experiment Settings

Dataset: To train the *side-channel attack models*, we constructed a CNN classification dataset following [17]. We generated 4,000 random DNN computation graphs by varying layer (e.g., depth, types) and dimension parameters (e.g., input/output channels, kernel sizes), using 3,200 for training and 800 for validation/testing. The number of CNN layers was sampled from [5,20] and fully connected (FCN) layers from [1,5]. At each layer, the block type was chosen as Conv2D (60%), ResNet (20%), or MobileNet (20%), with dimensions sampled jointly. After each convolution, pooling, batch normalization, and an activation (σ, $\tanh$, or ReLU) were included uniformly at random. Finally, at least two FCN layers ending with softmax were appended. All DNNs were optimized with Apache TVM, executed on a GPU, and CUDA kernel traces were collected via Nsight Compute[1] for training and evaluation.

To train and validate our *triplet networks*, we randomly selected 96 CNNs from the random dataset, along with four well-known benchmark models: VGG-11 [30], VGG-13 [30], MobileNet [31], and ResNet-20 [1], all designed for the 10-class classification problem in the CIFAR-10 dataset. Among these, 80 random CNNs were used for training, while the remaining 16 random CNNs combined with the four benchmarks constituted the testing set. To generate obfuscation samples, each DNN is randomly obfuscated 100 times at both the architecture level (i.e., $N_{\text{arc}} = 100$ in Algorithm 1) and the scheduling level (i.e., $N_{\text{sch}} = 100$ in Algorithm 4). For both levels, we obtained 80×100 candidates for training and 20×100 candidates for testing, sampled as positive and negative pairs. At the architecture level, after applying the sampling algorithm in Algorithm 2, the training set included 7080 sample pairs, and the testing set included 1920 sample pairs. At the scheduling level, after applying Algorithm 5, the training set included 7130 sample pairs, and the testing set included 1920 sample pairs. For both Algorithm 2 and Algorithm 5, we set $a_{\text{num}} = 7$, $b_{\text{num}} = 10$, and $c_{\text{num}} = 50$.

[1] A proprietary tool maintained by NVIDIA Corporation for CUDA kernel profiling, similar to NVPROF.

Evaluation Metrics: Following prior work [12], [16], [17], we use the ler to evaluate the accuracy of layer sequences recovered by attack models:

$$ler = \frac{1}{N}\sum_{i}\frac{dis_{edit}(\tilde{\mathbf{la}}_i, \mathbf{la}_i)}{|\mathbf{la}_i|} \tag{2}$$

where $\tilde{\mathbf{la}}_i$ is the predicted layer sequence of model M_i, $\mathbf{la}_i$ is the original ground-truth sequence of length $|\mathbf{la}_i|$, and dis_{edit} is the edit distance [32], i.e., the minimum number of insertions, deletions, and substitutions needed to transform one sequence into the other. A larger average ler indicates stronger defense. We report the average ler over two attack models, DeepSniffer+ and TPAM.

To evaluate obfuscated DNN performance, we measure single-inference latency (seconds). Specifically, we run 100 inferences and report the average latency. To fairly compare obfuscation time, all methods receive the same inputs for each victim DNN: 1) the PyTorch model M_i, and 2) the TVM-optimized best configuration of the original architecture, including tuning history. We deploy the original DNN with this configuration to obtain the baseline latency and set the latency constraint accordingly, then measure the time each method takes to generate its final obfuscated solution. We also use an accuracy metric to select architectures and hyperparameters for the triplet networks. For each anchor with positive and negative samples, the network outputs two distances. If the negative-anchor distance is larger than the positive-anchor distance, we count a true positive and true negative; otherwise, a false positive and false negative.

Baselines: We compare our method with two SOTA baselines, `NeurObf+` [16] and `DNNAlias+` [24]. For fairness, we align both baselines to both levels where possible.

- `NeurObf+` uses the ler from [12] as the GA fitness under latency constraints. We keep its architecture- and scheduling-level components unchanged, but use the average ler of two attack models as the fitness score (Section II-A).
- `DNNAlias+` minimizes the standard deviation of side-channel features in a GA-based architecture-level obfuscation method under latency constraints. We keep its evaluator unchanged and add the scheduling-level obfuscation from [16] to its GA framework.

Implementation Details: All DNN models (datasets, attack methods, baselines, and our method) are implemented using PyTorch on an NVIDIA GeForce RTX 3090 GPU. Attack-model hyper-parameters are tuned for best performance. Baselines use 20 generations with a population size of 16. For the scheduling-level triplet network, key hyper-parameters include 3 multi-resolution blocks, output dimensions 16 and 64 for "configs" and "workloads" in the selective embedding block, 64 for task types, and a final embedding size $n_{sch} = 64$. For the architecture-level triplet network, the anchor LSTM has an output dimension of 32, others 128, with a final embedding size $n_{arc} = 64$.

B. Overall Performance

We compare our method with two SOTAs across three axes: security (ler), performance (latency), and obfuscation efficiency (minutes). We evaluate the test set under latency constraints from strict to relaxed (0.001-0.003 s). For example, in Table II, the first "latency" column shows clean (unobfuscated) DNN latencies ranging from 0.0003 s to 0.0012 s. We use a 0.002 s constraint in Table II since it exceeds all clean latencies and thus applies to every test model.

As shown in Table II, our method achieves an order-of-magnitude faster obfuscation, reducing generation time by 90% compared to prior SOTA approaches (minutes per DNN vs. hours). This efficiency improvement is practically important because obfuscation may need to be repeated across model variants or deployment targets. When each attempt requires hours of on-device evaluation, iterative deployment becomes impractical. CLOb-DNN substantially reduces candidate screening time, making obfuscation more practical in real-world settings. On average, our approach requires 46 minutes to obfuscate each DNN, whereas `NeurObf+` and `DNNAlias+` take 8 hours 3 minutes, and 7 hours 41 minutes per DNN, respectively. In total, our method spends 14 hours 34 minutes to obfuscate 19 victim DNNs, while `NeurObf+` and `DNNAlias+` consume 152 hours 57 minutes, and 145 hours 59 minutes, respectively. For every DNN in the testing set, our method consistently exhibits the shortest running time, effectively reducing obfuscation time by 90% compared to the SOTA methods.

From a security perspective, as illustrated in first column of Table II (ler part), our method boosts LER from 0.341 (no obfuscation) to 2.885 on average (7× improvement), achieving the highest LER among baselines and best security on 80% of the tested DNNs while remaining comparable in the rest. Compared to `NeurObf+` and DNNAlias+, which achieve average ler of 2.461 and 1.667 respectively, our method maintains the highest ler. Specifically, our obfuscation solutions achieve the best security effect for approximately 80% of the DNNs in the testing set (with the best results marked in bold). For the remaining 20% of cases, `NeurObf+` attains the highest ler in most instances (except for model VGG13, where `DNNAlias+` outperforms), but our method still achieves comparable ler values (reaching 86% of the best solutions' security level on average). For example, for Model 3, our method attains $ler = 2.969$ compared to the best solution's $ler = 3.047$, achieving 97% of the performance, and importantly, does so with significantly higher efficiency.

In terms of latency, SOTA methods generally outperform CLOb-DNN but remain within an acceptable range; on average, CLOb-DNN incurs 0.0005 s and 0.0002 s higher latency than `NeurObf+` and `DNNAlias+`, respectively. This gap arises because SOTA methods evaluate each candidate on the target device to obtain real latency. `NeurObf+` maximizes ler under a latency budget, while `DNNAlias+` minimizes a fitness score based on standard deviation; both penalize over-budget solutions, requiring repeated on-device evaluation and thus longer execution time. In contrast, our method enforces latency via backtracking (Algorithm 3) and limits scheduling obfuscation (Algorithm 6), treating latency as a constraint while maximizing security. In some cases (e.g., Models 3, 6, and 16), `NeurObf+` achieves higher ler and lower latency but at significantly higher time cost, indicating potential improvement directions for our method.

TABLE II
OVERALL PERFORMANCE OF OUR FRAMEWORK CLOB-DNN COMPARED TO NO-OBFUSCATION AND SOTAS WITH A LATENCY CONSTRAINT OF **0.002 S** (BEST RESULTS ARE MARKED AS BOLD), "No-Obf" MEANS WITHOUT ANY OBFUSCATION (MARKED AS ITALIC)

ModelID	$ler\uparrow$				latency↓ ($<0.002s$)				time↓		
	No-obf	NeurObf+	DNNAlias+	CLOb-DNN	No-obf	NeurObf+	DNNAlias+	CLOb-DNN	NeurObf+	DNN-Alias+	CLOb-DNN
1	*0.276*	2.586	1.982	**3.017**	*0.000408*	**0.00059**	0.000654	0.00103	7h45min	5h27min	**42min**
2	*0.277*	1.585	1.543	**1.851**	*0.001247*	**0.001561**	0.001931	0.00187	13h46min	12h44min	**59min**
3	*0.516*	**3.047**	2.469	2.969	*0.000568*	**0.001012**	0.001033	0.001261	8h38min	10h40min	**1h01min**
4	*0.238*	1.775	0.875	**1.975**	*0.000812*	**0.001463**	0.001997	0.0019	10h55min	9h36min	**53min**
5	*0.293*	1.776	1.103	**3.190**	*0.000351*	0.000532	**0.000455**	0.000915	5h35min	5h50min	**43min**
6	*0.25*	**3.5**	1.06	2.67	*0.000709*	**0.001382**	0.001992	0.001993	5h42min	4h05min	**29min**
7	*0.355*	1.936	1.887	**2.097**	*0.000542*	**0.000821**	0.001117	0.000943	6h44min	7h20min	**38min**
8	*0.5*	2.565	1.629	**2.887**	*0.000662*	**0.000875**	0.001582	0.001157	9h23min	7h10min	**49min**
9	*0.419*	2.973	2.162	**3.270**	*0.000586*	**0.001125**	0.001203	0.001582	9h13min	10h15min	**1h07min**
10	*0.263*	2.526	1.632	**3.105**	*0.000490*	**0.00057**	0.001011	0.00197	5h27min	4h36min	**31min**
11	*0.412*	2.691	2.147	**2.735**	*0.000544*	**0.00105**	0.001157	0.0018	10h21min	11h7min	**59min**
12	*0.391*	2.283	1.934	**3.283**	*0.000363*	**0.000578**	0.000940	0.001004	4h13min	5h13min	**35min**
13	*0.325*	2.45	1.025	**2.875**	*0.000559*	**0.000855**	0.001427	0.001474	5h03min	4h52min	**40min**
14	*0.347*	2.556	1.417	**3.069**	*0.000855*	**0.001375**	0.001960	0.001882	11h34min	9h54min	**59min**
15	*0.438*	2.297	1.406	**2.688**	*0.000613*	**0.000905**	0.001765	0.001841	8h50min	7h26min	**51min**
16	*0.298*	**2.0**	0.940	1.786	*0.001048*	**0.001772**	0.001987	0.00183	10h47min	10h57min	**57min**
vgg11	*0.315*	3.018	1.944	**3.055**	*0.000639*	0.001234	0.001221	0.001555	6h37min	6h10min	**49min**
vgg13	*0.306*	2.935	**3.129**	2.597	*0.000678*	0.001514	**0.001476**	0.001867	8h41min	9h	**49min**
resnet20	*0.267*	2.267	1.4	**5.7**	*0.000372*	**0.000459**	0.000517	0.000554	3h43min	3h37min	**15min**
average	*0.341*	2.461	1.667	**2.885**	*0.000634*	**0.001035**	0.001337	0.001503	8h3min	7h41min	**46min**
total	-	-	-	-	-	-	-	-	152h57min	145h59min	**14h34min**

TABLE III
OVERALL PERFORMANCE OF OUR FRAMEWORK CLOB-DNN COMPARED TO NO-OBFUSCATION AND SOTAS WITH A LATENCY CONSTRAINT OF **0.001 S** (BEST RESULTS ARE MARKED AS BOLD), "No-Obf" MEANS WITHOUT ANY OBFUSCATION (MARKED AS ITALIC)

ModelID	$ler\uparrow$				latency↓ ($<0.001s$)				time↓		
	No-obf	NeurObf+	DNNAlias+	CLOb-DNN	No-obf	NeurObf+	DNNAlias+	CLOb-DNN	NeurObf+	DNN-Alias+	CLOb-DNN
1	*0.276*	**2.586**	1.741	2.328	*0.000408*	**0.000594**	0.000953	0.000862	7h45min	7h29min	**1h4min**
3	*0.517*	2.406	1.672	**2.703**	*0.000568*	0.000963	0.000993	**0.000962**	10h10min	10h21min	**1h20min**
4	*0.238*	N.A.	0.650	**0.775**	*0.000812*	N.A.	0.000999	**0.000920**	10h26min	8h43min	**1h15min**
5	*0.293*	1.776	1.569	**3.0**	*0.000351*	**0.000532**	0.000671	0.000650	5h35min	5h52min	**46min**
6	*0.25*	**2.8**	1.0	1.361	*0.000709*	0.000998	0.000998	**0.000904**	5h18min	4h33min	**44min**
7	*0.355*	1.936	1.274	**2.226**	*0.000542*	**0.000821**	0.000991	0.000876	6h44min	7h5min	**48min**
8	*0.5*	**2.565**	1.339	2.468	*0.000662*	**0.000875**	0.000998	0.000909	9h23min	8h34min	**1h3min**
9	*0.419*	2.5	1.595	**2.784**	*0.000586*	**0.000929**	0.000993	0.000976	10h21min	10h27min	**1h30min**
10	*0.263*	**2.526**	2.211	**2.526**	*0.000490*	**0.00057**	0.000843	0.000646	5h27min	5h2min	**37min**
11	*0.412*	**2.706**	1.912	2.368	*0.000544*	0.000986	0.000994	**0.000870**	12h17min	11h26min	**1h12min**
12	*0.391*	2.281	1.435	**2.283**	*0.000363*	**0.000578**	0.000999	0.000695	4h13min	4h57min	**46min**
13	*0.325*	2.45	**2.9**	2.625	*0.000559*	**0.000855**	0.000985	0.000857	5h03min	5h5min	**45min**
14	*0.347*	1.403	**1.5**	1.236	*0.000855*	0.000998	0.000999	**0.000939**	10h51min	10h45min	**1h27min**
15	*0.438*	2.297	1.641	**2.328**	*0.000613*	0.000905	**0.000850**	0.000877	8h50min	7h45min	**1h**
vgg11	*0.315*	2.093	1.778	**2.611**	*0.000639*	0.000903	0.000993	0.000999	6h10min	6h15min	**1h3min**
vgg13	*0.306*	2.097	1.081	**2.161**	*0.000678*	0.000988	0.000999	**0.000951**	7h20min	7h27min	**1h14min**
resnet20	*0.267*	1.767	1.267	**5.633**	*0.000372*	0.000621	0.000568	0.000638	3h43min	3h34min	**17min**
average	*0.348*	2.262	1.562	**2.436**	*0.000574*	**0.000820**	0.000931	0.000855	7h37min	7h22min	**59min**
total	-	-	-	-	-	-	-	-	129h36min	125h20min	**16h51min**

We also evaluate each method under a strict latency constraint (i.e., 0.001 s), as shown in Table III (Model 2 and 16 are excluded since their clean latency is already larger than the budget, as such there is no room for obfuscation), our method still has the highest efficiency for each victim DNN. Even with a stringent 0.001 s latency constraint, our method remains the most efficient: obfuscating each DNN in 1 h on average (5-10 minutes for some backtracking cases) and reducing total time by 87% compared to SOTAs (>120 hours). In addition, under the rigorous constraint 0.001 s, for some DNNs which require the backtracking algorithm, we successfully generate valid architecture solutions with small time consumption, i.e., 5-10 minutes.

From a security standpoint, under the strict 0.001 s latency budget, our method achieves the highest average LER (2.436, 6× over no-obf) and the best LER on 15/19 DNNs, while remaining more robust: unlike NeurObf+, which can fail to produce any feasible solution (e.g., Model 4) whereas our backtracking reliably finds valid low-latency obfuscations. Notably, NeurObf+ may fail under the 0.001 s latency constraint; for example, for Model 4, it runs for over 10 hours without producing any solution that meets the latency requirement (indicated as N.A. in Table III). This occurs because all initial populations have latencies exceeding 0.001 s, and adjusting the reward function by incorporating $t - t_{\text{budget}}$ in the denominator does not help the algorithm escape local optima. In contrast, our backtracking algorithm directly retracts obfuscation knobs to reduce latency, efficiently generating valid solutions.

Under the relaxed 0.003 s latency constraint (Table IV), our method remains the most efficient, requiring only 59 minutes per DNN on average, versus over 8 hours for both baselines (88% reduction). It also achieves the highest average ler (3.155vs. 2.543 and 1.680), improving security by about 9× over no obfuscation, and attains the best ler on nearly 84% of the tested DNNs while satisfying the latency constraint for all models. The average latency of our obfuscated models is 0.001588 s, still within the 0.003 s budget and only 0.000563 s higher than the lowest-latency baseline. By comparison, NeurObf+ achieves the lowest latency in most cases and the highest ler on four models, but at much higher time cost, while DNNAlias+ is inferior to our method in both ler and efficiency.

TABLE IV
OVERALL PERFORMANCE OF OUR FRAMEWORK COMPARED TO NO-OBFUSCATION AND SOTAS WITH A LATENCY CONSTRAINT OF **0.003 S** (BEST RESULTS IN BOLD), "No-Obf" MEANS WITHOUT ANY OBFUSCATION (MARKED AS ITALIC)

ModelID	*ler* ↑				latency↓ (< 0.003s)				time↓		
	No-obf	NeurObf+	DNNAlias+	CLOb-DNN	No-obf	NeurObf+	DNNAlias+	CLOb-DNN	NeurObf+	DNN-Alias+	CLOb-DNN
1	*0.276*	2.155	1.397	**3.138**	*0.000408*	0.000764	**0.000562**	0.001461	7h33min	7h23min	**1h6min**
2	*0.277*	1.638	1.064	**1.979**	*0.001247*	**0.001650**	0.001860	0.002189	13h29min	13h10min	**1h23min**
3	*0.516*	2.781	2.719	**3.641**	*0.000568*	**0.001010**	0.001111	0.001587	10h18min	10h19min	**1h11min**
4	*0.238*	1.85	0.663	**2.313**	*0.000812*	**0.001767**	0.003	0.002092	10h13min	9h43min	**1h8min**
5	*0.293*	2.241	1.310	**3.310**	*0.000351*	0.000655	**0.000534**	0.001244	5h58min	5h25min	**50min**
6	*0.25*	**4.028**	1.889	3.583	*0.000709*	**0.001301**	0.002472	0.001602	5h28min	5h10min	**37min**
7	*0.355*	2.097	1.306	**2.355**	*0.000542*	**0.000864**	0.000919	0.001212	6h56min	6h58min	**49min**
8	*0.5*	2.5	2.387	**2.903**	*0.000662*	**0.000900**	0.001159	0.001093	7h34min	8h59min	**1h3min**
9	*0.419*	2.216	1.784	**3.581**	*0.000586*	**0.000923**	0.001320	0.001502	10h48min	9h56min	**1h24min**
10	*0.263*	2.605	1.316	**3.158**	*0.000490*	**0.000695**	0.000950	0.000901	5h13min	5h46min	**38min**
11	*0.412*	2.382	1.559	**2.485**	*0.000544*	**0.000937**	0.001041	0.001423	11h46min	10h44min	**1h14min**
12	*0.391*	**2.957**	1.261	**2.957**	*0.000363*	0.000540	**0.000434**	0.001209	5h23min	5h12min	**46min**
13	*0.325*	3.075	2.8	**3.725**	*0.000559*	0.00115	0.001202	0.002046	5h48min	5h2min	**45min**
14	*0.347*	**3.25**	2.069	2.861	*0.000855*	**0.001141**	0.002742	0.001985	11h13min	10h55min	**1h17min**
15	*0.438*	2.453	1.813	**3.344**	*0.000613*	0.000940	**0.000915**	0.001561	7h59min	7h26min	**1h5min**
16	*0.298*	**2.333**	1.060	1.881	*0.001048*	**0.001299**	0.002795	0.002351	11h59min	11h20min	**52min**
vgg11	*0.315*	2.889	1.963	**3.444**	*0.000639*	**0.000964**	0.001122	0.001705	7h24min	6h23min	**1h3min**
vgg13	*0.306*	2.968	2.194	**3.129**	*0.000678*	**0.001520**	0.001544	0.002368	8h39min	9h27min	**1h9min**
resnet20	*0.267*	1.9	1.367	**6.167**	*0.000372*	**0.00045**	0.000521	0.000641	3h39min	3h40min	**18min**
average	*0.341*	2.543	1.680	**3.155**	*0.000634*	**0.001025**	0.001379	0.001588	8h16min	8h3min	**59min**
total	-	-	-	-	-	-	-	-	157h20min	152h48min	**18h38min**

TABLE V
OVERALL PERFORMANCE OF MOBILENET UNDER LATENCY CONSTRAINTS: **0.0035 S**, **0.004 S** AND **0.005 S** (BEST RESULTS ARE MARKED AS BOLD), "No-Obf" MEANS WITHOUT ANY OBFUSCATION (MARKED AS ITALIC)

MobileNet		*ler*	latency	time
	No-obf	*1.167*	*0.003152*	
0.0035s	NeurObf+	2.167	0.003191	11h36min
	DNNAlias+	1.5	0.0035	12h11min
	CLOb-DNN	**2.25**	**0.002884**	**1h12min**
0.004s	NeurObf+	2.306	0.003477	13h36min
	DNNAlias+	1.75	0.003996	12h11min
	CLOb-DNN	**2.444**	**0.003289**	**1h13min**
0.005s	NeurObf+	2.139	0.004930	16h3min
	DNNAlias+	1.361	0.004848	14h25min
	CLOb-DNN	**2.5**	**0.003095**	**1h13min**

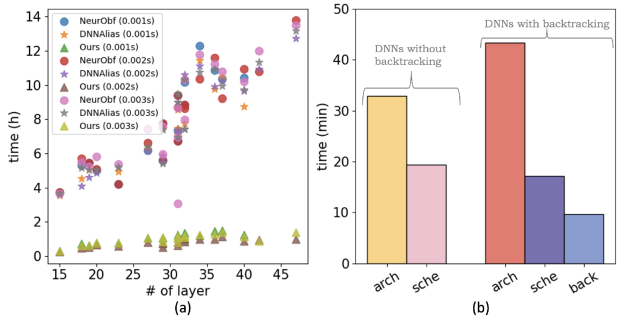


Fig. 9. Time consumption distribution of CLOb-DNN and baselines. (a) Time consumption (in hours) and layer number distribution of each victim DNN with our proposed method, NeurObf+ [16] and DNNAlias+ [24]. (b) Average time consumption (in minutes) of each step in our method, DNNs are divided into two groups, which do not and do require (under latency constraint 0.001 s) our back-tracking algorithm.

For MobileNet, whose clean latency is already 0.003152 s, we evaluate CLOb-DNN under three latency constraints: 0.0035 s, 0.004 s, and 0.005 s. Due to MobileNet's heavier workload, both NeurObf+ and DNNAlias+ suffer out-of-memory termination after 7 GA generations and require checkpoint-based restart, so we limit the experiments to 10 generations. Even then, both baselines require 12–16 hours (Table V). In contrast, CLOb-DNN consistently outperforms both baselines in mean *ler*, latency, and time cost. It needs just over 1 h to generate an obfuscation that satisfies the latency constraints, achieves the shortest latency, and improves *ler* by 92%–114% over the baselines across different constraints. Because it evaluates only a small number of candidates on the target device, it also uses much less memory and is more flexible for DNNs with heavy workloads.

Fig. 9 compares the time consumption of CLOb-DNN and the two baselines. Although obfuscation is offline, generation time still matters when the per-model cost is high, especially for deep models, multiple deployment variants, or frequently updated models. In Fig. 9(a), both NeurObf+ (circles) and DNNAlias+ (stars) show rapidly increasing time cost with layer count, whereas our method (triangles) remains stable, indicating better scalability. Fig. 9(b) shows that, without backtracking, architecture- and scheduling-level obfuscation take about 30 and 20 minutes, respectively; with backtracking, they take about 45, 20, and 10 minutes. Only 8 DNNs require backtracking under the 0.001 s constraint, and none under 0.002 s or 0.003 s, indicating limited additional cost even under strict latency budgets.

C. Evaluation of Triplet Networks

To evaluate the two triplet networks independently, we first assess the architecture-level network. For each test DNN, we randomly generate 100 obfuscated architectures, compute their distances to the anchor using the trained triplet network, and omit configuration obfuscation to isolate architecture effects. We then deploy them on the target device, collect side-channel traces, and compute *ler* using attack models.

Fig. 10 illustrates the relationship between the distances produced by the architecture-level triplet network and the mean *ler* values of 100 architecture candidates generated for each test DNN. In each subfigure, the x-axis shows the distances computed by our triplet network, and the y-axis reports the corresponding average *ler* values. The architecture-level triplet network was trained for 50 epochs. On the test triplets ⟨anchor,

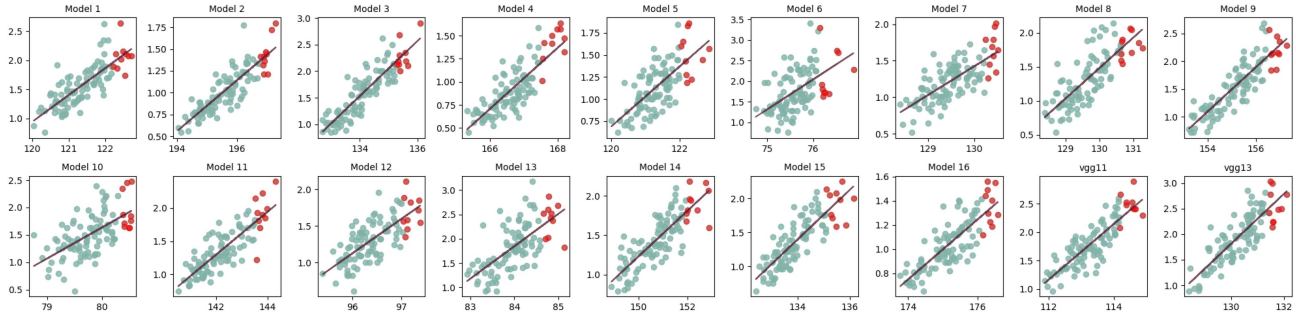


Fig. 10. Case studies of architecture-level obfuscation, each circle is an obfuscated candidate, the x-axis is distance in embedding space learned by our triplet network (i.e. Fig. 4) while the y-axis represents the average ler. The learned architecture level triplet network achieves an **accuracy=0.7528**.

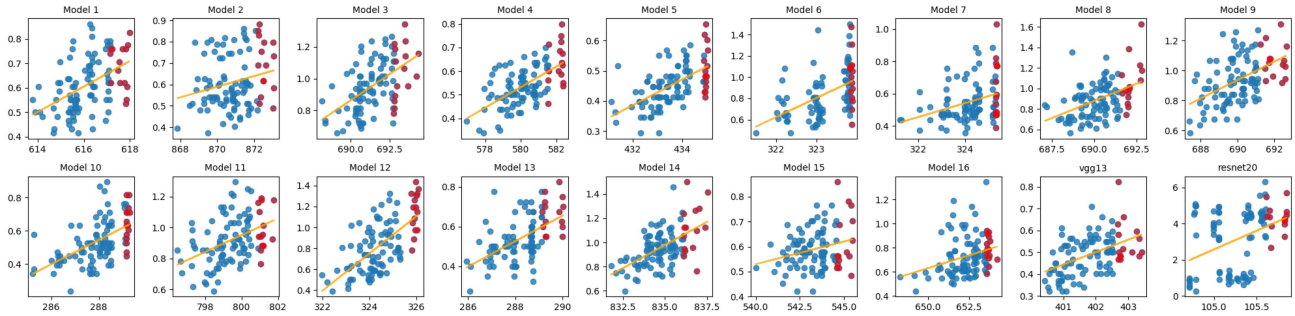


Fig. 11. Case studies of scheduling-level obfuscation, each circle represents an obfuscated candidate, with the x-axis indicating the distance in the embedding space computed by our scheduling-level triplet network, and the y-axis showing the corresponding average ler. The scheduling-level triplet network **achieves an accuracy of 0.687**.

positive, negative⟩, it achieves a ranking accuracy of 0.7528, where a triplet is counted as correct if the positive sample is embedded closer to the anchor than the negative sample. Since random ordering corresponds to an accuracy of 0.5, this result indicates that the learned embedding provides a meaningful ranking signal for candidate filtering rather than a final defense metric. The total training time is 6 hours for 25 epochs, and the inference time is 40 s for 1920 test samples, corresponding to 0.0208 s per sample. A clear positive correlation can be observed between the computed distances and the resulting ler values, indicating that the learned embedding space is effective for identifying high-quality obfuscated architectures. To visualize this trend, a least-squares linear fit (i.e., a first-degree polynomial) is superimposed on each subfigure. However, the candidate with the largest distance is not always the one with the highest ler. Therefore, rather than selecting only the single furthest candidate, we retain the top-p_a furthest candidates and evaluate them empirically during the online testing stage (see Fig. 4). The red circles in Fig. 10, which denote the top-p_a candidates with $p_a = 10$, show that our method can effectively identify high-quality obfuscation solutions while requiring only a small number of empirical evaluations.

Similarly, for each victim DNN, we generated 100 obfuscated configurations without architecture-level obfuscation. The scheduling-level triplet network projects these configurations into the embedding space, computing all 100 distances within seconds. Trained for 50 epochs, it attains 0.687 accuracy. Total training time is 17 hours; inference over 1920 test samples takes 126 seconds (0.0656 s/sample). Each subfigure shows a clear positive correlation between distance and ler, highlighted by the linear fit. Red circles mark the furthest p_c=15 candidates, which contain many qualified solutions.

Fig. 11 presents case studies of scheduling-level obfuscation across multiple models. Embedding distances show a strong positive correlation with average ler, indicating that the learned triplet metric reliably distinguishes more robust schedules. Together with Fig. 10, the results further reveal consistent clustering of robust candidates that combine architectural redundancy with scheduling irregularity, thereby providing empirical support for the effectiveness of the surrogate. The scheduling-level triplet network achieves an accuracy of 0.687.

D. Evaluation on Training Efficiency and Adversarial Attack

This subsection shows that obfuscation hinders attackers from obtaining performance-equivalent substitutes or mounting efficient adversarial attacks. We consider a strong attacker with access to the training and testing data. We sample 10 ImageNet classes [2], 1000 images each (700 train, 300 test). The attacker applies SCA to infer layers in obfuscated VGG11, VGG13, and ResNet20 via side-channel traces and an attack model. For dimensional details, the attacker randomly samples combinations, then trains candidate stolen models on the available data to select the best substitute. A higher LER indicates that the attacker-recovered layer sequence deviates more substantially from the ground-truth architecture, and therefore requires more

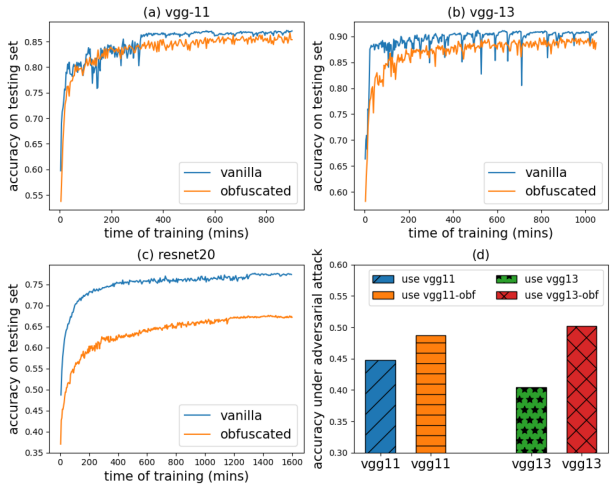


Fig. 12. The illustration of training time and corresponding test accuracy on both vanilla and obfuscated DNNs; and the accuracy under adversarial attack.

architectural corrections before a performance-equivalent substitute can be obtained. As shown in Fig. 12(a)–(c), which illustrate training efficiency (x-axis: training time in minutes on GPU, y-axis: accuracy on testing set), the stolen obfuscated DNN (orange line) consistently underperforms compared to directly training the original DNN (blue line). This result reflects the practical consequence of higher LER: the attacker faces increased cost and difficulty in reconstructing a usable substitute model. These results complement LER with a direct evaluation of downstream attack success: even when the attacker uses the recovered architecture under the same training budget, the obfuscated models yield lower substitute-model accuracy and weaker adversarial transferability. Notably, for ResNet20, the accuracy gap reaches 20%, showing that obfuscation substantially increases attack cost of IP theft even against strong attackers. We further consider an adversarial transfer setting where the attacker uses the stolen substitute to craft white-box adversarial examples against the victim model. Following [33], we evaluate VGG11 and VGG13, as the stolen ResNet20 substitute does not converge satisfactorily. As shown in Fig. 12(d), for VGG11, accuracy drops from 0.85 to 0.44 without obfuscation (blue bar), but remains at 0.48 with obfuscation (orange bar) due to the mismatch between the substitute and original architectures. A similar trend holds for VGG13, where accuracy drops from 0.90 to 0.40 without obfuscation, but remains at 0.50 with obfuscation, yielding a 10% robustness gain.

While our prototype is implemented on Apache TVM, the proposed obfuscation framework is not TVM-specific. Our framework encompasses both scheduling-level and architecture-level obfuscation, which are loosely coupled. At the scheduling layer, our method formulates obfuscation as a multi-objective tuning problem: instead of minimizing only latency as in existing works (e.g., [1,2]), we optimize a side-channel leakage-aware cost function over the same schedule primitives (e.g., tiling, loop reordering). TVM's tuning logs are used merely as stable, machine-readable artifacts of schedule search, and similar mechanisms exist in other compilers. For instance, Ansor exposes per-operator schedule candidates that can be re-scored with additional objectives, while AITemplate stores best-config JSON files that are reused in subsequent compilations. Therefore, our approach is transferable to any system that provides i) an exposed search space of schedule primitives, and ii) a mechanism to deploy chosen configurations. Moreover, the architecture-level obfuscation is fully independent of TVM and can be applied within high-level frameworks such as PyTorch or MXNet. Based on these observations, we recommend that future compiler: a) expose schedule primitives and provide stable, externalized tuning records to enable deployment of specific configurations and external scoring, b) support pluggable multi-objective cost models (latency, security, energy), and c) support seedable stochastic sampling with early constraint-based pruning. These features would provide the necessary foundations for compiler designers to incorporate obfuscation as a first-class optimization objective.

V. Related Work

This section reviews existing works on SCAs and mitigation techniques. The work in [7] utilized memory and timing side-channels to extract CNN architectures on accelerators, but required complete memory traces. [8] relied on timing side-channels and assumed that the DNN belonged to known templates, limiting applicability for unknown DNNs. [9] used GPU context-switching side-channels to train LSTM predictors, which required access to the training phase, making it impractical. Several studies [10], [11], [14], [34] leveraged cache side-channels to recover DNN information but relied on strong assumptions, such as monitoring deep learning framework function calls or requiring specific GEMM implementations. [13] recovered DNN architectures using a classifier and consistency optimization based on integer programming via EM side-channels, which required attaching a low-cost induction sensor to the GPU. [12] and [17] represent the SOTA methods for recovering DNN layer sequences. [12] utilized EM side-channel attacks without assuming prior knowledge of the victim model and trained an LSTM-based network to predict layer sequences based on architectural hints. However, their method's performance degraded significantly when DNNs were optimized by TVM for efficient inference. [16] refines DeepSniffer [12] by targeting only complex layers (convolution, pooling, softmax), noting that TVM often fuses convolution, BatchNorm, and ReLU into one kernel; nonetheless, performance remains suboptimal. [17] proposes a two-phase attack to steal optimized DNNs: an attention-LSTM to learn intra-/inter-layer fusion patterns, followed by a phase modeling runtime temporal correlations between operations and layers for precise layer-sequence prediction.

Along with development of SCA methods, various mitigation techniques have been proposed. However, some approaches involve modifying memory access mechanisms [19], [20], [21] or altering computer architecture layers [22], leading to high performance overhead and incompatibility with existing hardware. Apache Tensor Virtual Machine (TVM) [23], while primarily designed to accelerate DNN inference, also offers potential as an SCA countermeasure, providing compatibility with diverse hardware and deep learning frameworks. However, TVM alone is insufficient, as demonstrated by [16] and [17], where the latter

developed an attack framework capable of accurately recovering layer sequences even after TVM optimization. [16] introduced a comprehensive tool for DNN obfuscation at both the script and back-end levels, inspiring several works [24], [25], [26], [35]. However, [16] suffers from inefficiency, while subsequent studies [24], [25], [35] focused only on architecture-level obfuscation. [24] also faced high execution times, and [25] was inapplicable to unseen DNN architectures. [35] showed that the obfuscated layer sequence generated by [16] could be reverted to the original using a translation model, assuming the adversary has the obfuscated layer sequence. Specifically, they treated the obfuscated layer sequence as a sentence in a source language and the original layer sequence as the target, establishing a one-to-one correspondence between them. However, current SOTA SCA models cannot precisely steal obfuscated sequences, especially after backend optimization/obfuscation, which typically results in over 50% prediction error. Therefore, this assumption is impractical due to multiple factors, including genetic algorithm settings, backend obfuscation strategies, and varying SCA methods. [26] focused exclusively on scheduling-level obfuscation for efficiency, neglecting architecture-level obfuscation and failing to ensure adherence to specific latency constraints.

VI. Conclusion

We propose a contrastive learning framework for efficient, robust DNN obfuscation against SCAs. Using triplet networks at both architecture and scheduling levels, our method cuts obfuscation time by 90% by minimizing repeated on-device deployments, while meeting latency constraints and strengthening defenses against advanced attacks. Unlike prior work, it attains stable protection without costly backend changes and covers diverse threat models. Future work will explore a unified framework that jointly models architecture-level and scheduling-level obfuscation, while extending our method to tighter latency budgets, broader DNN architectures, and additional attack vectors.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[2] O. Russakovsky et al., "Imagenet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, pp. 211–252, 2015.

[3] A. Vaswani et al., "Attention is all you need," *in Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 6000–6010.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. 17th Annu. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 4171–4186.

[5] A. Zeroual, F. Harrou, A. Dairi, and Y. Sun, "Deep learning methods for forecasting COVID-19 time-series data: A comparative study," *Chaos, Solitons Fractals*, vol. 140, 2020, Art. no. 110121.

[6] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," 2017, *arXiv:1707.01926*.

[7] W. Hua, Z. Zhang, and G. E. Suh, "Reverse engineering convolutional neural networks through side-channel information leaks," in *Proc. 55th ACM/ESDA/IEEE Des. Automat. Conf.*, 2018, pp. 1–6.

[8] V. Duddu, D. Samanta, D. V. Rao, and V. E. Balas, "Stealing neural networks via timing side channels," 2018, *arXiv:1812.11720*.

[9] J. Wei, Y. Zhang, Z. Zhou, Z. Li, and M. A. Al Faruque, "Leaky DNN: Stealing deep-learning model secret with GPU context-switching side-channel," in *Proc. 50th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2020, pp. 125–137.

[10] S. Hong et al., "Security analysis of deep neural networks operating in the presence of cache side-channel attacks," 2018, *arXiv:1810.03487*.

[11] M. Yan, C. W. Fletcher, and J. Torrellas, "Cache telepathy: Leveraging shared resource attacks to learn DNN architectures," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 2003–2020.

[12] X. Hu et al., "Deepsniffer: A DNN model extraction framework based on learning architectural hints," in *Proc. 25th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2020, pp. 385–399.

[13] H. T. Maia, C. Xiao, D. Li, E. Grinspun, and C. Zheng, "Can one hear the shape of a neural network?: Snooping the GPU via magnetic side channel," in *Proc. 31st USENIX Secur. Symp.*, vol. 22, 2022, pp. 4383–4400.

[14] H. Wang, S. M. Hafiz, K. Patwari, C.-N. Chuah, Z. Shafiq, and H. Homayoun, "Stealthy inference attack on DNN via cache-based side-channel attacks," in *Proc. Des., Automat. Test Europe Conf. Exhib.*, 2022, pp. 1515–1520.

[15] C. Gongye, Y. Fei, and T. Wahl, "Reverse-engineering deep neural networks using floating-point timing side-channels," in *Proc. 57th ACM/IEEE Des. Automat. Conf.*, 2020, pp. 1–6.

[16] J. Li, Z. He, A. S. Rakin, D. Fan, and C. Chakrabarti, "Neurobfuscator: A full-stack obfuscation tool to mitigate neural architecture stealing," in *Proc. IEEE Int. Symp. Hardware Oriented Secur. Trust*, 2021, pp. 248–258.

[17] Y. Sun, G. Jiang, X. Liu, P. He, and S.-K. Lam, "Layer sequence extraction of optimized DNNs using side-channel information leaks," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 43, no. 10, pp. 3102–3115, Oct. 2024.

[18] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, "PRADA: Protecting against DNN model stealing attacks," in *Proc. Euro. Symp. Secur. Privacy*, 2019, pp. 512–527.

[19] E. Stefanov et al., "Path oram: An extremely simple oblivious ram protocol," *J. ACM*, vol. 65, no. 4, pp. 1–26, 2018.

[20] S. Patel, G. Persiano, M. Raykova, and K. Yeo, "PanORAMa: Oblivious RAM with logarithmic overhead," in *Proc. IEEE 59th Annu. Symp. Foundations Comput. Sci.*, 2018, pp. 871–882.

[21] Y. Liu, D. Dachman-Soled, and A. Srivastava, "Mitigating reverse engineering attacks on deep neural networks," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI*, 2019, pp. 657–662.

[22] H. Nam, R. P. Pothukuchi, B. Li, N. S. Kim, and J. Torrellas, "Defensive ML: Defending architectural side-channels with adversarial obfuscation," 2023, *arXiv:2302.01474*.

[23] A. TVM, "Apache TVM documentation," 2020. [Online]. Available: https://tvm.apache.org/docs/index.html

[24] M. M. Ahmadi, L. Alrahis, O. Sinanoglu, and M. Shafique, "DNN-alias: Deep neural network protection against side-channel attacks via layer balancing," 2023, *arXiv:2303.06746*.

[25] T. Zhou, S. Ren, and X. Xu, "Obfunas: A neural architecture search-based DNN obfuscation approach," in *Proc. 41st IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2022, pp. 1–9.

[26] Y. Sun, S.-K. Lam, G. Jiang, and P. He, "Streamlining DNN obfuscation to defend against model stealing attacks," in *Proc. IEEE Int. Symp. Circuits Syst.*, 2024, pp. 1–5.

[27] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Symp. Operating Syst. Des. Implementation*, 18, 2018, pp. 578–594.

[28] S. Liu, Y. Wei, J. Chi, F. H. Shezan, and Y. Tian, "Side channel attacks in computation offloading systems with GPU virtualization," in *Proc. IEEE Secur. Privacy Workshops*, 2019, pp. 156–161.

[29] H. Naghibijouybari, A. Neupane, Z. Qian, and N. Abu-Ghazaleh, "Rendered insecure: GPU side channel attacks are practical," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 2139–2153.

[30] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. 3rd Int. Conf. Learn. Representations*, 2015, *arXiv:1409.1556*.

[31] A. G. Howard et al., "Mobilenets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[32] G. Navarro, "A guided tour to approximate string matching," *ACM Comput. Surv.*, vol. 33, no. 1, pp. 31–88, 2001.

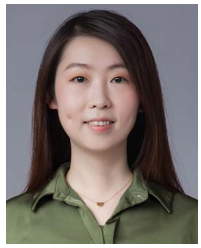
[33] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *Stat*, vol. 1050, no. 9, 2017, pp. 1–28.

[34] Y. Liu and A. Srivastava, "Ganred: GAN-based reverse engineering of DNNs via cache side-channel," in *Proc. ACM SIGSAC Conf. Cloud Comput. Secur. Workshop*, 2020, pp. 41–52.
[35] M. M. Ahmadi, L. Alrahis, A. Colucci, O. Sinanoglu, and M. Shafique, "Neurounlock: Unlocking the architecture of obfuscated deep neural networks," in *Proc. Int. Joint Conf. Neural Netw.*, 2022, pp. 1–10.

Yidan Sun received the BEng from the School of Computer Science and Technology, Tianjin University, China, and the PhD from the School of Computer Science and Engineering, Nanyang Technological University (NTU), Singapore. She is currently a research fellow with Imperial Global—Singapore, Imperial College London. She was a research fellow with NTU. Her research interests include AI security, Big Data analytics in transportation, spatio-temporal data analysis, and energy-efficient MPSoC design.

Guiyuan Jiang (Member, IEEE) received the BS degree in computer science and technology from Northwest University for Nationalities, the MEng degree in computer science and technology from Tianjin Polytechnic University, and the PhD degree in computer science and technology from Tianjin University (TJU). He was a research fellow with Nanyang Technological University, Singapore. He is currently an associate professor with the School of Computer Science and Technology, Ocean University of China. He has authored or coauthored more than 70 papers in venues such as *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Intelligent Transportation Systems*, *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, *IEEE Transactions on Information Forensics and Security*, *IEEE/ACM Transactions on Networking*, IJCAI, DATE, and SDM. His interests include traffic data analytics, public transport modeling and optimization, and ocean Big Data processing.

Jiayang Liu Jiayang Liu received the BS degree from Dalian Maritime University (DMU), in 2015, and the PhD degree from the University of Science and Technology of China (USTC), in 2020. He has been a postdoctoral researcher with the RIKEN Center for Advanced Intelligence Project since 2020. After that, he has been a research fellow with the National University of Singapore since 2022. He is currently a research fellow with Nanyang Technological University. His research interests include adversarial machine learning and information hiding.



Qian Sun received the BS degree from Tianjin University, and the PhD degree in computer science from Nanyang Technological University, Singapore, 2014. She was a research fellow with Fraunhofer Singapore from 2014 to 2017, and an associate professor with Tianjin University from 2017 to 2021. She is currently a professor with the School of Electronic and Information Engineering, Nanjing University of Information Science and Technology. Her current research interests include intelligent graphics and image processing, and deep learning.

Peilan He received the BM degree from the School of Economics and Management, Hainan University, the MEng degree from the School of Computer Science and Technology, Tianjin University, and the PhD degree from the School of Computer Science and Engineering, Nanyang Technological University, Singapore. She is currently a lecturer with the School of Computer Science and Technology, Ocean University of China. Her research interests include machine learning for combinatorial optimization, transportation Big Data analytics, and multimodal journey planning.

Siew Kei Lam (Senior Member, IEEE) received the BASc, MEng, and PhD degrees from the School of Computer Science and Engineering, NTU. He was a visiting research fellow with Imperial College London, the University of Warwick, and RWTH Aachen. He is currently an associate professor with the NTU's College of Computing and Data Science. He has authored or coauthored more than 200 papers in venues such as *IEEE Transactions on Computers*, *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, *IEEE Transactions on Circuits and Systems II: Express Briefs*, *IEEE Transactions on Circuits and Systems for Video Technology*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Intelligent Transportation Systems*, DATE, ASP-DAC, FPL, ICFPT, and HASP@ISCA. His research interests include develops custom computing for edge intelligence, targeting performance, energy efficiency, cost, reliability, and security, span secure and reliable edge systems, domain-specific AI architectures, visual edge intelligence for autonomy, and distributed intelligence for smart mobility. He was an associate editor for *IET Circuits, Devices and Systems*.